

CS 660: Data Mining For Decision Making

Lecture 5 (Week 2)

Varun Dutt

School of Computing and Electrical Engineering
School of Humanities and Social Sciences
Indian Institute of Technology Mandi, India



Scaling the Heights

WEKA (Witten, Frank, & Hall, 2011)

- In this class, we will be using an open-source data-mining tool called WEKA (<http://www.cs.waikato.ac.nz/ml/weka/>).
- Please download WEKA and install it on your laptop/computer (the most updated release is WEKA 3.6.12 at the time of writing this assignment).



First Step: Preparing the Input

- WEKA uses attribute-relation file format (ARFF) Format
- Example: Weather Dataset

Table 1.3 Weather Data with Some Numeric Attributes				
Outlook	Temperature	Humidity	Windy	Play
Sunny	85	85	false	no
Sunny	80	90	true	no
Overcast	83	86	false	yes
Rainy	70	96	false	yes
Rainy	68	80	false	yes
Rainy	65	70	true	no
Overcast	64	65	true	yes
Sunny	72	95	false	no
Sunny	69	70	false	yes
Rainy	75	80	false	yes
Sunny	75	70	true	yes
Overcast	72	90	true	yes
Overcast	81	75	false	yes
Rainy	71	91	true	no

ARFF Format

```
% ARFF file for the weather data with some numeric features
%
@relation weather

@attribute outlook { sunny, overcast, rainy }
@attribute temperature numeric
@attribute humidity numeric
@attribute windy { true, false }
@attribute play? { yes, no }

@data
%
% 14 instances
%
sunny, 85, 85, false, no
sunny, 80, 90, true, no
overcast, 83, 86, false, yes
rainy, 70, 96, false, yes
rainy, 68, 80, false, yes
rainy, 65, 70, true, no
overcast, 64, 65, true, yes
sunny, 72, 95, false, no
sunny, 69, 70, false, yes
rainy, 75, 80, false, yes
sunny, 75, 70, true, yes
overcast, 72, 90, true, yes
overcast, 81, 75, false, yes
rainy, 71, 91, true, no
```

ARFF Format

Figure 2.2 shows an ARFF file for the weather data in Table 1.3, the version with some numeric features. Lines beginning with a % sign are comments. Following the comments at the beginning of the file are the name of the relation (*weather*) and a block defining the attributes (*outlook, temperature, humidity, windy, play?*). Nominal attributes are followed by the set of values they can take on, enclosed in curly braces. Values can include spaces; if so, they must be placed within quotation marks. Numeric values are followed by the keyword *numeric*.

Although the weather problem is to predict the class value *play?* from the values of the other attributes, the class attribute is not distinguished in any way in the data file. The ARFF format merely gives a dataset; it does not specify which of the attributes is the one that is supposed to be predicted. This means that the same file can be used for investigating how well each attribute can be predicted from the others, or it can be used to find association rules or for clustering.

ARFF Format: other attributes

As well as nominal and numeric attributes, exemplified by the weather data, the ARFF format has three further attribute types: string attributes, date attributes, and relation-valued attributes. String attributes have values that are textual. Suppose you have a string attribute that you want to call *description*. In the block defining the attributes it is specified like this:

```
@attribute description string
```

Then, in the instance data, include any character string in quotation marks (to include quotation marks in your string, use the standard convention of preceding each one by a backslash, \). Strings are stored internally in a string table and represented by their address in that table. Thus, two strings that contain the same characters will have the same value.

ARFF Format: date attribute

Date attributes are strings with a special format and are introduced like this (for an attribute called *today*):

```
@attribute today date
```

Weka uses the ISO-8601 combined date and time format *yyyy-MM-dd'THH:mm:ss* with four digits for the year, two each for the month and day, then the letter *T* followed by the time with two digits for each of hours, minutes, and seconds.¹ In the data section of the file, dates are specified as the corresponding string representation of the date and time—for example, 2004-04-03T12:00:00. Although they are specified as strings, dates are converted to numeric form when the input file is read. Dates can also be converted internally to different formats, so you can have absolute time-stamps in the data file and use transformations to forms such as time of day or day of the week to detect periodic behavior.

ARFF Format: relational attribute

enced instances. For example, a relation-valued attribute called *bag*, with a value that is a dataset that has the same structure as the weather data but without the *play* attribute, can be specified like this:

```
@attribute bag relational
  @attribute outlook { sunny, overcast, rainy }
  @attribute temperature numeric
  @attribute humidity numeric
  @attribute windy { true, false }
@end bag
```

The *@end bag* indicates the end of the nested attribute block. Figure 2.3 shows an ARFF file for a multi-instance problem based on the weather data. In this case, each example is made up of an identifier value, two consecutive instances from the original weather data, and a class label.

Each value of the attribute is a string that encapsulates two weather instances separated by the `\n` character (which represents an embedded new line). This might be appropriate for a game that lasts two days. A similar dataset might be used for games that last for an indeterminate number of days (e.g., first-class cricket takes three to five days). Note, however, that in multi-instance learning the *order* in which

ARFF Format: relational attribute

```
% Multiple instance ARFF file for the weather data
%
@relation weather

@attribute bag_ID { 1, 2, 3, 4, 5, 6, 7 }
@attribute bag relational
    @attribute outlook { sunny, overcast, rainy }
    @attribute temperature numeric
    @attribute humidity numeric
    @attribute windy { true, false }
@end bag
@attribute play? { yes, no }

@data
%
% seven "multiple instance" instances
%
1, "sunny, 85, 85, false\nsunny, 80, 90, true", no
2, "overcast, 83, 86, false\nrainy, 70, 96, false", yes
3, "rainy, 68, 80, false\nrainy, 65, 70, true", yes
4, "overcast, 64, 65, true\nsunny, 72, 95, false", yes
5, "sunny, 69, 70, false\nrainy, 75, 80, false", yes
6, "sunny, 75, 70, true\novercast, 72, 90, true", yes
7, "overcast, 81, 75, false\nrainy, 71, 91, true", yes
```

Naïve Bayes Classifier Demo in WEKA

Table 4.2 Weather Data with Counts and Probabilities

Outlook			Temperature			Humidity			Windy			Play	
	yes	no		yes	no		yes	no		yes	no	yes	no
sunny	2	3	hot	2	2	high	3	4	false	6	2	9	5
overcast	4	0	mild	4	2	normal	6	1	true	3	3		
rainy	3	2	cool	3	1								
sunny	2/9	3/5	hot	2/9	2/5	high	3/9	4/5	false	6/9	2/5	9/14	5/14
overcast	4/9	0/5	mild	4/9	2/5	normal	6/9	1/5	true	3/9	3/5		
rainy	3/9	2/5	cool	3/9	1/5								

Table 4.3 A New Day

Outlook	Temperature	Humidity	Windy	Play
Sunny	cool	high	true	?

Laplace Estimator in WEKA: That is why results are different from the tool and text

For example, the upper part of Table 4.2 shows that for *play = yes*, *outlook* is *sunny* for two examples, *overcast* for four, and *rainy* for three, and the lower part gives these events probabilities of 2/9, 4/9, and 3/9, respectively. Instead, we could add 1 to each numerator, and compensate by adding 3 to the denominator, giving probabilities of 3/12, 5/12, and 4/12, respectively. This will ensure that an attribute value that occurs zero times receives a probability which is nonzero, albeit small. The strategy of adding 1 to each count is a standard technique called the *Laplace estimator* after the great eighteenth-century French mathematician Pierre Laplace. Although it works well in practice, there is no particular reason for adding 1 to the counts: We could instead choose a small constant μ and use

$$\frac{2 + \mu/3}{9 + \mu}, \quad \frac{4 + \mu/3}{9 + \mu}, \quad \text{and} \quad \frac{3 + \mu/3}{9 + \mu}$$

Activities

- Read Singhal, 2011
- Do Assignment 1

Thank you!

Comments and Questions most welcome!